

Diseño de una arquitectura LSTM para la predicción de demanda energética

Valery Nickol Rosero Molina y Johan David Delgado

Inteligencia Artificial

Dr. Cristian Camilo Ordoñez Quintero

7 de octubre de 2026

Repositorio del proyecto: <https://github.com/Valery-Rosero/Red-Neuronal-App>

Aplicación web: <https://red-neuronal-app.vercel.app>

Resumen

Este trabajo diseña, implementa, compara y despliega redes neuronales recurrentes de memoria a corto y largo plazo (LSTM) para predecir la demanda eléctrica de la hora siguiente a partir de información horaria multivariada (demanda, clima, precio y calendario) del periodo 2025-2026. Primero se auditó la calidad del conjunto de datos y se aplicaron reglas de limpieza que no inventan valores: los datos inválidos se marcaron como faltantes y se descartaron las ventanas que los contenían. La serie se dividió cronológicamente en entrenamiento, validación y prueba, y se normalizó solo con parámetros de entrenamiento. Se compararon tres arquitecturas (una LSTM base, una LSTM profunda y una CNN-LSTM) con ventanas de 12, 24 y 48 horas. El modelo seleccionado por validación, la LSTM base con ventana de 12 horas, obtuvo en un periodo nunca visto un MAE de 19,86 MW, un RMSE de 24,99 MW, un MAPE de 3,43 % y un R^2 de 0,904. Esto representa un 44 % menos de error que un pronóstico de persistencia y un 15 % menos que una regresión lineal. No se encontró sobreajuste relevante. Los experimentos de control mostraron que Dropout y la detención temprana fueron determinantes para generalizar, y que aumentar el número de neuronas no mejoró los resultados. Finalmente, el modelo se desplegó en una aplicación web que valida las entradas con las mismas reglas del entrenamiento.

Palabras clave: LSTM, predicción de demanda eléctrica, series de tiempo, redes neuronales recurrentes, aprendizaje profundo

Diseño de una arquitectura LSTM para la predicción de demanda energética

La operación de un sistema eléctrico depende de anticipar cuánta energía se va a consumir. Una predicción precisa de la demanda de la próxima hora permite programar la generación, reducir los costos de reserva y evitar desbalances. La demanda tiene una estructura temporal marcada: depende de lo ocurrido en las horas anteriores, de los ciclos diario y semanal, y de variables externas como el clima, el precio de la energía y los días festivos.

Las redes neuronales recurrentes de memoria a corto y largo plazo (*Long Short-Term Memory*, LSTM) fueron propuestas para aprender dependencias temporales largas, que las redes recurrentes simples no logran capturar por el desvanecimiento del gradiente (Goodfellow et al., 2016; Hochreiter y Schmidhuber, 1997). Este informe documenta el diseño, la implementación, la comparación y el despliegue de arquitecturas LSTM para predecir la demanda eléctrica de la hora siguiente a partir de una ventana de n horas consecutivas de información multivariada.

El desarrollo completo (código, cuaderno de Jupyter y modelos entrenados) está disponible en el repositorio del proyecto (<https://github.com/Valery-Rosero/Red-Neuronal-App>), y el modelo desplegado puede usarse en la aplicación web (<https://red-neuronal-app.vercel.app>). Todo se implementó en Python con TensorFlow y Keras (Chollet et al., 2015), pandas, NumPy y scikit-learn (Pedregosa et al., 2011). Las cifras, tablas y figuras de este informe provienen directamente de la ejecución del cuaderno.

Objetivos

El objetivo general es diseñar, implementar, comparar y justificar arquitecturas LSTM para predecir la demanda eléctrica de la siguiente hora a partir de información histórica multivariada. Los objetivos específicos son:

1. Auditar la calidad del conjunto de datos y definir reglas de limpieza justificadas que no inventen valores.
2. Preparar los datos sin fuga de información futura: orden cronológico, separación cronológica en entrenamiento, validación y prueba, y normalización con parámetros calculados solo sobre entrenamiento.

3. Diseñar tres arquitecturas (LSTM base, LSTM profunda y una propuesta propia) y evaluarlas con ventanas de 12, 24 y 48 horas.
4. Evaluar los modelos con MAE, MSE, RMSE, MAPE y R^2 , y analizar el sobreajuste y la capacidad de generalización.
5. Responder las preguntas de análisis del taller con evidencia experimental.
6. Desplegar el modelo seleccionado en una aplicación web.

Descripción del problema y del conjunto de datos

El problema consiste en predecir la demanda de la hora $t + 1$ a partir de una ventana de n horas consecutivas que termina en la hora t , con $n \in \{12, 24, 48\}$. El conjunto de datos (`dataset_demanda_energia_LSTM_2025_2026.csv`) contiene registros horarios del 1 de enero de 2025 al 31 de diciembre de 2026, con 17 550 filas y 15 columnas, que se describen en la Tabla 1.

Tabla 1

Variables del conjunto de datos

Variable	Tipo	Descripción
<code>timestamp</code>	Fecha y hora	Instante de la observación (resolución horaria)
<code>demanda_mw</code>	Continua	Demanda eléctrica observada en la hora t (MW)
<code>temperatura_c</code>	Continua	Temperatura (°C)
<code>humedad_pct</code>	Continua	Humedad relativa (%)
<code>viento_kmh</code>	Continua	Velocidad del viento (km/h)
<code>radiacion_wm2</code>	Continua	Radiación solar (W/m ²)
<code>precipitacion_mm</code>	Continua	Precipitación (mm)
<code>precio_kwh</code>	Continua	Precio de la energía
<code>hora</code> , <code>dia_semana</code> , <code>mes</code>	Calendario	Hora (0-23), día de la semana (0 = lunes) y mes (1-12)
<code>fin_semana</code> , <code>festivo</code>	Binaria	Indicadores de fin de semana y de día festivo
<code>periodo_dia</code>	Texto	Madrugada, mañana, tarde o noche
<code>demanda_objetivo</code>	Objetivo	Demanda de la hora $t + 1$ (variable a predecir)

Se verificó que `demanda_objetivo` en la fila t coincide con `demanda_mw` en la fila $t + 1$ en el 99,8 % de las horas. Las diferencias aparecen solo donde `demanda_mw` presenta picos anómalos, lo que confirma que el objetivo es la demanda de la hora siguiente y que esos picos son errores de medición.

Calidad de los datos y reglas de limpieza

Problemas detectados

La auditoría inicial encontró los cinco tipos de problemas que anuncia el enunciado del taller (ver Tabla 2).

Tabla 2

Problemas de calidad detectados en el conjunto de datos

Problema	Evidencia
Desorden temporal	Las filas no están ordenadas por <code>timestamp</code>
Duplicados	30 filas exactamente repetidas
Valores faltantes	70 nulos en temperatura, humedad, viento y precio; 1 en el objetivo (la última hora)
Inconsistencias de texto	<code>periodo_dia</code> con 8 variantes, como <code>TARDE</code> , <code>Mañana</code> o <code>noche</code>
Anomalías	Humedad de hasta 129,7 %, viento y precio negativos, temperaturas de $-23,5^{\circ}\text{C}$ y $54,7^{\circ}\text{C}$, y picos de demanda de hasta 1 776 MW, aunque el objetivo nunca supera 927 MW

Reglas de limpieza

El principio rector, exigido por el taller, es no inventar valores. Ningún dato se imputa, interpola ni corrige: lo inválido se marca como faltante (`NaN`) y se excluye toda ventana que lo contenga. La Tabla 3 resume las reglas aplicadas y su justificación.

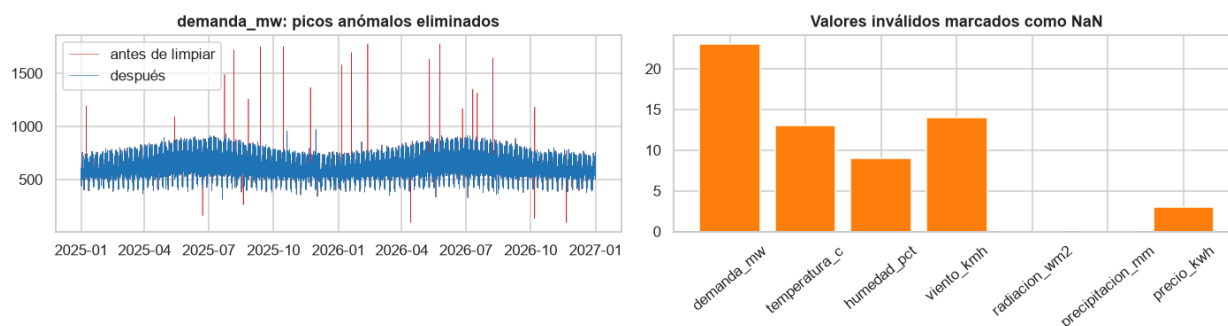
Tabla 3*Reglas de limpieza aplicadas y su justificación*

Regla	Descripción	Justificación
R1	Normalizar el texto de <code>periodo_dia</code> (sin espacios y en minúsculas)	Son la misma categoría escrita de forma distinta; se pasa de 8 a 4 valores
R2	Eliminar los duplicados exactos	Repetir una observación la sobrepondera y podría filtrar información entre conjuntos
R3	Ordenar por <code>timestamp</code> antes de crear secuencias	Una secuencia solo tiene sentido en orden temporal
R4	Verificar que haya una fila por hora	Tras R2 y R3 la serie queda completa: 17 520 horas sin huecos
R5	Valores fuera de rango físico → <code>NaN</code>	Humedad entre 0 y 100 %; viento, radiación, precipitación y precio no negativos; temperatura entre -10 y 45 °C
R6	<code>demanda_mw</code> fuera de [300, 1000] MW → <code>NaN</code>	El objetivo, que está limpio, varía entre 314 y 927 MW; los picos de 1 100-1 776 MW no aparecen en él
R7	Descartar las ventanas con algún <code>NaN</code>	Evita imputar, a costa de perder algunas ventanas
R8	Descartar la última fila	El 31 de diciembre de 2026 a las 23:00 no tiene hora siguiente

Con las reglas R5 y R6 se marcaron como inválidos 23 valores de demanda, 13 de temperatura, 9 de humedad, 14 de viento y 3 de precio. Ningún valor de radiación ni de precipitación quedó fuera de rango. La Figura 1 muestra el efecto de la limpieza sobre la demanda.

Figura 1

Efecto de la limpieza sobre la demanda y valores marcados como inválidos



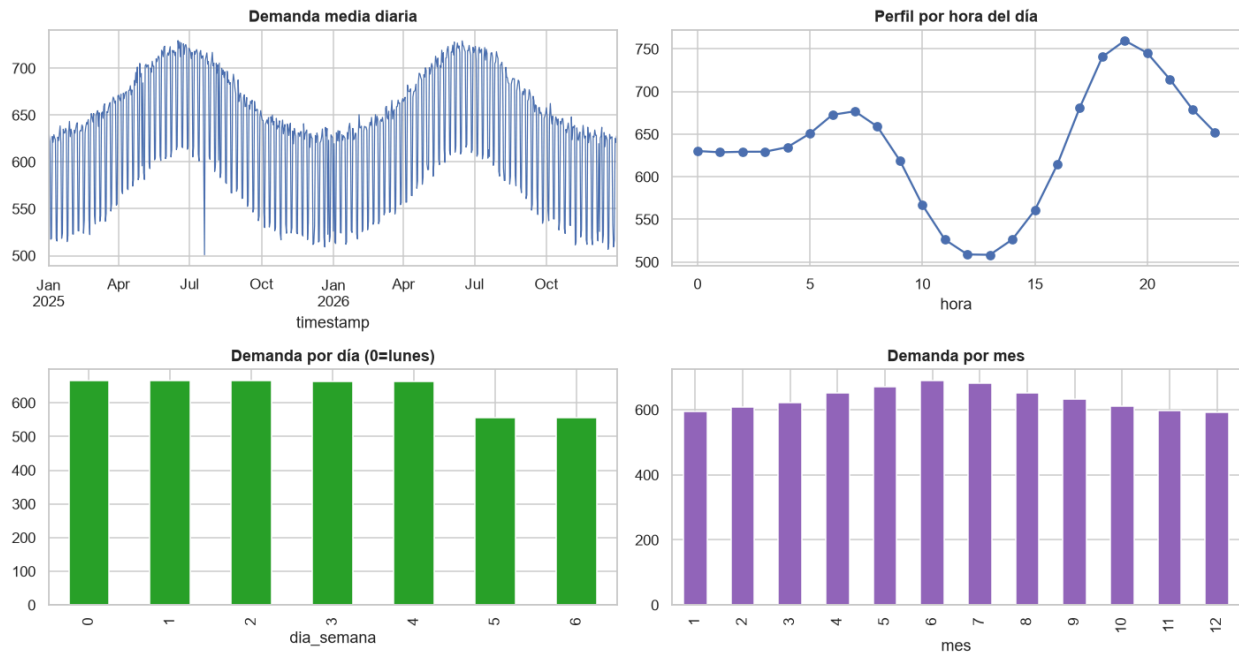
Nota. A la izquierda, demanda antes (rojo) y después (azul) de la limpieza; a la derecha, número de valores marcados como inválidos por variable.

Análisis exploratorio

La demanda presenta un perfil diario muy marcado, con un mínimo en la madrugada y picos en la mañana y al final de la tarde. También disminuye los fines de semana y varía a lo largo del año (ver Figura 2).

Figura 2

Demanda media diaria y perfiles medios por hora, día de la semana y mes

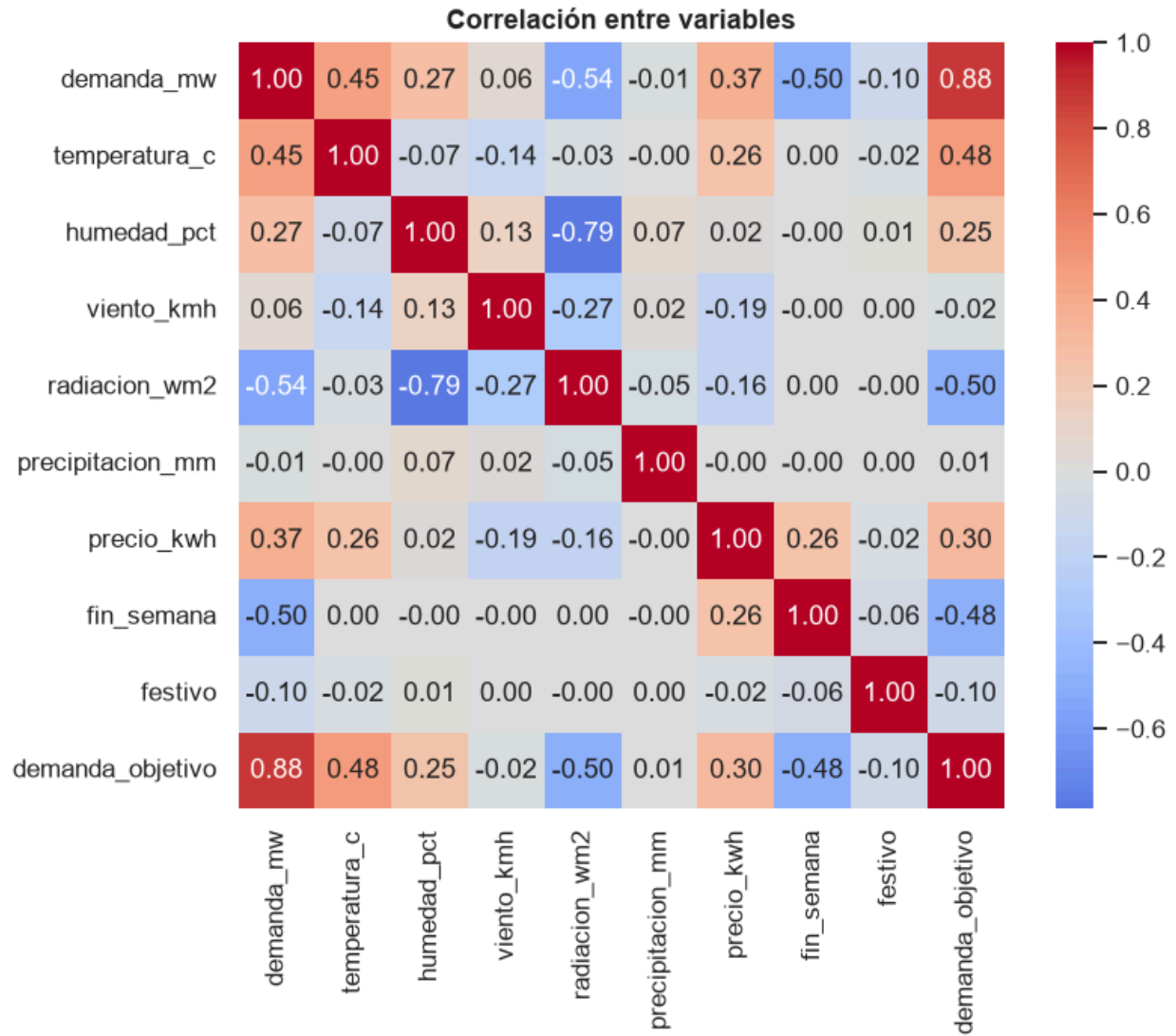


Nota. En el panel del día de la semana, 0 corresponde al lunes.

Las variables más correlacionadas con la demanda de la hora siguiente son la demanda actual ($r = 0,88$), la radiación ($r = -0,50$), el fin de semana ($r = -0,49$), la temperatura ($r = 0,48$) y el precio ($r = 0,30$). El viento y la precipitación casi no se relacionan con la demanda ($|r| < 0,02$), como se observa en la Figura 3.

Figura 3

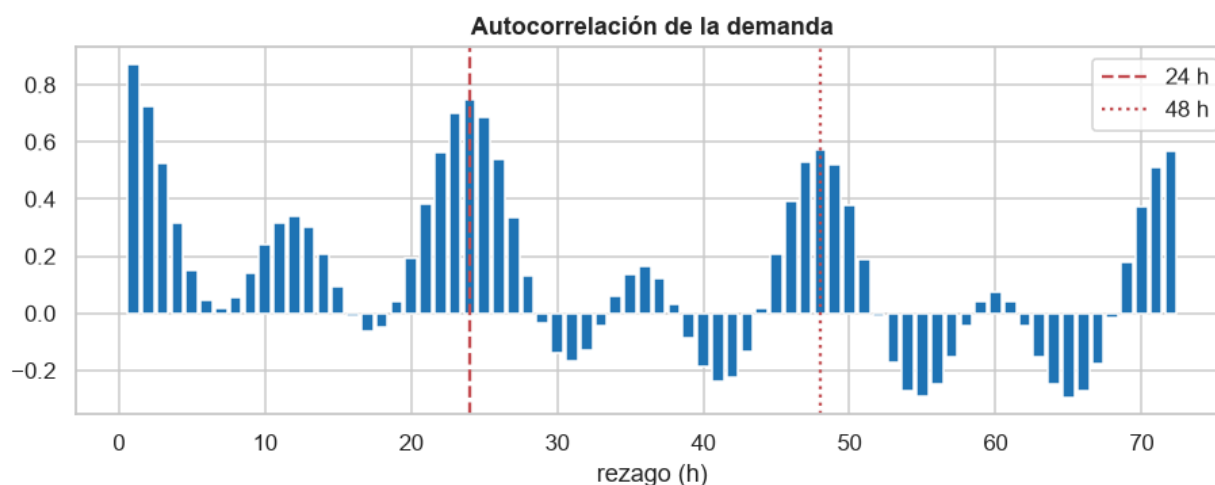
Matriz de correlación entre las variables continuas, los indicadores de calendario y el objetivo



La autocorrelación de la Figura 4 confirma la dependencia temporal y el ciclo de 24 horas, lo que justifica el uso de ventanas de varias horas y de modelos con memoria.

Figura 4

Autocorrelación de la demanda para rezagos de 1 a 72 horas



Nota. Las líneas rojas marcan los rezagos de 24 y 48 horas.

Metodología

Variables de entrada

Cada hora de la ventana se describe con 15 variables:

- Siete continuas: demanda, temperatura, humedad, viento, radiación, precipitación y precio. La demanda pasada es una entrada legítima porque se conoce en el instante t .
- Seis cíclicas: la hora, el día de la semana y el mes, codificados como seno y coseno. Así, las 23:00 quedan cerca de las 00:00 y diciembre cerca de enero, algo que una codificación numérica directa no logra.
- Dos binarias: fin de semana y festivo.

La variable `periodo_dia` se limpió pero se excluyó del modelo, porque es una función exacta de `hora` y no aporta información nueva. La variable `demanda_objetivo` nunca se usa como entrada.

Separación cronológica y normalización

En las series de tiempo, la evaluación debe respetar el orden temporal: el modelo se entrena con el pasado y se evalúa con el futuro (Hyndman y Athanasopoulos, 2021). Por eso los datos se dividieron por orden temporal en proporción 70/15/15, sin `train_test_split` aleatorio (ver Tabla 4).

Tabla 4

Separación cronológica del conjunto de datos

Conjunto	Periodo	Horas	Uso
Entrenamiento	01-01-2025 00:00 a 26-05-2026 23:00	12 264	Ajustar los pesos
Validación	27-05-2026 00:00 a 13-09-2026 11:00	2 628	Detención temprana y selección del modelo
Prueba	13-09-2026 12:00 a 31-12-2026 23:00	2 628	Evaluación final, sin uso en ninguna decisión

La normalización z (media 0 y desviación estándar 1) usa la media y la desviación estándar calculadas únicamente sobre el conjunto de entrenamiento. Esos mismos parámetros se aplican a validación, a prueba y a la aplicación web.

Construcción de las ventanas

Para cada hora t , la entrada es la matriz de las horas $[t - n + 1, \dots, t]$ ($n \times 15$) y el objetivo es la demanda de $t + 1$. Cada ventana se asigna al conjunto que corresponde a su hora t , de modo que ningún objetivo de validación o prueba se observa durante el entrenamiento. El cuaderno incluye verificaciones automáticas de que no existe fuga temporal.

Cada valor inválido elimina las n ventanas que lo contienen, por lo que la ventana de 48 horas conserva menos muestras (ver Tabla 5). Para comparar las ventanas de forma justa, la evaluación principal usa un conjunto común de prueba: los 1 042 instantes válidos para la ventana de 48 horas.

Tabla 5

Ventanas disponibles por conjunto tras descartar las que contienen valores inválidos

Ventana	Entrenamiento	Validación	Prueba
12 h	9 686	2 051	2 093
24 h	7 526	1 605	1 664
48 h	4 504	964	1 042
Conjunto común	4 504	964	1 042

Nota. El conjunto común está formado por los instantes válidos para la ventana de 48 horas, que también son válidos para las de 12 y 24 horas.

Cumplimiento de las condiciones del taller

La Tabla 6 relaciona cada condición del enunciado con la forma en que se cumplió.

Tabla 6*Cumplimiento de las condiciones del taller*

Condición	Cómo se cumplió
No utilizar información futura como entrada	La entrada solo contiene las horas $[t - n + 1, \dots, t]$; el objetivo $(t + 1)$ se excluye y hay verificaciones automáticas
Ordenar cronológicamente antes de crear secuencias	Regla R3, aplicada antes de construir cualquier ventana
Separar entrenamiento, validación y prueba cronológicamente	Cortes por fecha en proporción 70/15/15
No utilizar <code>train_test_split</code> aleatorio	La división se hace por índice temporal
Justificar las reglas de limpieza	Reglas R1 a R8, cada una con su justificación
No inventar valores para corregir anomalías	Lo inválido pasa a <code>NaN</code> y se descarta la ventana; la aplicación web aplica la misma regla
Normalizar solo con parámetros de entrenamiento	Media y desviación estándar calculadas sobre entrenamiento
Comparar mínimo tres arquitecturas LSTM	LSTM base, LSTM profunda y CNN-LSTM
Experimentar con ventanas de 12, 24 y 48 horas	Tres arquitecturas por tres ventanas: nueve modelos
Evaluar con MAE, MSE, RMSE y R^2	Se reportan además el MAPE y las épocas
Analizar el sobreajuste y la generalización	Apartados de sobreajuste y de experimentos de control
Gráficas de pérdida, real frente a predicha, error y comparación	Figuras de los apartados de entrenamiento y resultados
Desplegar el modelo en una aplicación web	Aplicación pública en Vercel (ver el apartado de despliegue)

Diseño de las arquitecturas

Se diseñaron tres alternativas, descritas en la Tabla 7. El número de parámetros de una LSTM no depende de la longitud de la ventana.

Tabla 7*Arquitecturas evaluadas*

Modelo	Capas	Parámetros	Justificación
A. LSTM base	LSTM (32) → Dense (1)	6 177	Línea de referencia: la red recurrente más simple. Si un modelo más complejo no la supera, su complejidad no se justifica
B. LSTM profunda	LSTM (64) → Dropout (0, 2) → LSTM (32) → Dropout (0, 2) → Dense (16, ReLU) → Dense (1)	33 441	Las capas apiladas aprenden representaciones jerárquicas, y el Dropout controla el sobreajuste que trae la mayor capacidad
C. CNN-LSTM (propuesta)	Conv1D (32, k = 3, causal) → LSTM (48) → Dropout (0, 2) → Dense (16, ReLU) → Dense (1)	17 825	La convolución causal detecta patrones locales y filtra ruido antes de la LSTM, que modela la dependencia larga; cada paso solo ve horas anteriores

Nota. Los parámetros corresponden a una entrada de 15 variables por hora.

Para que la comparación fuera justa, todos los modelos se entrenaron con la misma configuración:

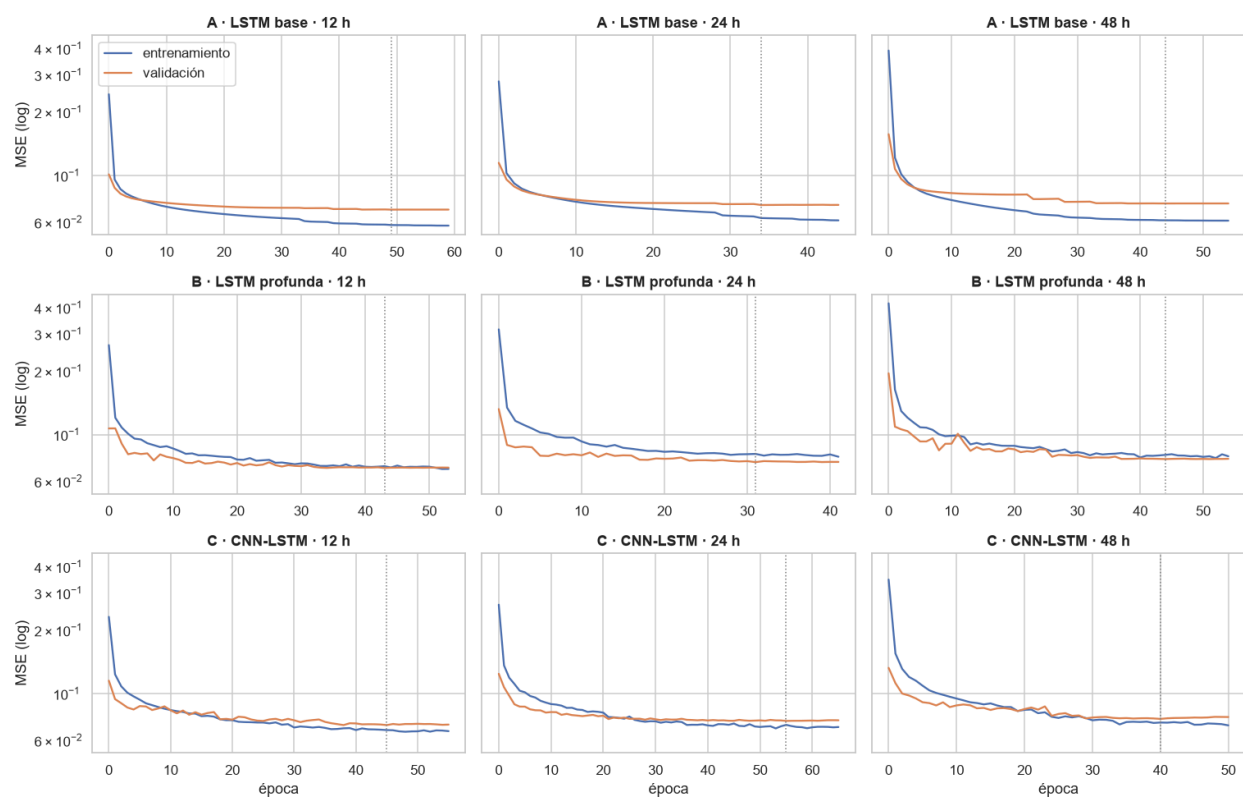
- Pérdida de error cuadrático medio (MSE) y optimizador Adam (Kingma y Ba, 2015) con una tasa de aprendizaje inicial de 0,001.
- Lotes de 64 ejemplos y un máximo de 80 épocas.
- Detención temprana (*early stopping*) sobre la pérdida de validación, con una paciencia de 10 épocas y restauración de los mejores pesos (Prechelt, 1998).
- Reducción de la tasa de aprendizaje a la mitad si la validación no mejora en 4 épocas.
- Semilla aleatoria fija (42), para que los resultados sean reproducibles.

Entrenamiento

La Figura 5 muestra las curvas de pérdida de los nueve modelos y la Tabla 8, las épocas de entrenamiento de cada uno.

Figura 5

Curvas de pérdida de entrenamiento y validación de los nueve modelos



Nota. Pérdida MSE sobre datos normalizados, en escala logarítmica. La línea punteada marca la época con la mejor validación.

Tabla 8*Épocas de entrenamiento de cada modelo*

Modelo	Ventana	Épocas ejecutadas	Mejor época	MSE de validación	Tiempo (s)
A. LSTM base	12 h	60	50	0,0689	50
A. LSTM base	24 h	45	35	0,0725	45
A. LSTM base	48 h	55	45	0,0737	59
B. LSTM profunda	12 h	54	44	0,0695	106
B. LSTM profunda	24 h	42	32	0,0740	99
B. LSTM profunda	48 h	55	45	0,0765	137
C. CNN-LSTM	12 h	56	46	0,0710	65
C. CNN-LSTM	24 h	66	56	0,0743	90
C. CNN-LSTM	48 h	51	41	0,0759	72

Nota. La detención temprana detiene el entrenamiento 10 épocas después de la mejor y restaura los pesos de esa época. El MSE de validación se calcula sobre datos normalizados en la mejor época; el tiempo corresponde a una CPU.

Todos los modelos convergieron entre 42 y 66 épocas, sin llegar al máximo de 80. La arquitectura A, que no tiene Dropout, es la única en la que la pérdida de entrenamiento sigue bajando mientras la de validación se estabiliza, lo que indica un sobreajuste leve. En B y C la pérdida de validación queda igual o por debajo de la de entrenamiento. No es un error: el Dropout solo está activo al calcular la pérdida de entrenamiento y se desactiva al validar (Srivastava et al., 2014).

Resultados

Métricas

Se usaron cinco métricas:

- El error absoluto medio (MAE), que es el error típico en MW.
- El error cuadrático medio (MSE), que penaliza más los errores grandes.
- Su raíz cuadrada (RMSE), expresada en MW.
- El error porcentual absoluto medio (MAPE).

- El coeficiente de determinación (R^2), que indica la fracción de la variabilidad de la demanda que explica el modelo.

Como referencia se usó el pronóstico de persistencia, que supone que la próxima hora será igual a la actual (Hyndman y Athanasopoulos, 2021).

Tabla de resultados

La Tabla 9 presenta las métricas de los nueve modelos sobre el conjunto común de prueba.

Tabla 9

Resultados sobre el conjunto común de prueba

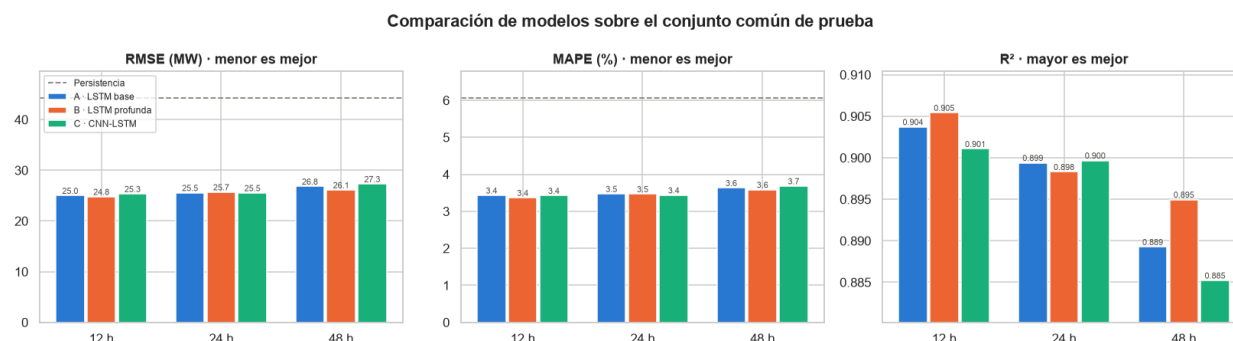
Modelo	Ventana	MAE	MSE	RMSE	MAPE	R^2	Épocas
A. LSTM base ✓	12 h	19,86	624,6	24,99	3,43 %	0,9037	60
A. LSTM base	24 h	20,12	652,6	25,55	3,47 %	0,8994	45
A. LSTM base	48 h	21,06	718,4	26,80	3,63 %	0,8893	55
B. LSTM profunda	12 h	19,43	613,5	24,77	3,37 %	0,9054	54
B. LSTM profunda	24 h	20,05	659,7	25,69	3,47 %	0,8983	42
B. LSTM profunda	48 h	20,68	681,4	26,10	3,58 %	0,8950	55
C. CNN-LSTM	12 h	19,75	641,8	25,33	3,42 %	0,9011	56
C. CNN-LSTM	24 h	19,86	650,8	25,51	3,43 %	0,8997	66
C. CNN-LSTM	48 h	21,28	744,6	27,29	3,68 %	0,8852	51
Persistencia	—	35,50	1 963,3	44,31	6,07 %	0,6974	—

Nota. 1 042 instantes entre el 26 de septiembre y el 31 de diciembre de 2026. MAE y RMSE en MW; MSE en MW². ✓ = modelo seleccionado por validación.

Los nueve modelos superan ampliamente a la persistencia: obtienen un RMSE de 24,8 a 27,3 MW frente a 44,3 MW, es decir, entre un 38 y un 44 % menos de error, y un R^2 de 0,89 a 0,91 frente a 0,70. La ventana de 12 horas es la mejor en las tres arquitecturas y la de 48 horas, la peor. Las diferencias entre arquitecturas son pequeñas (como máximo 0,6 MW de RMSE con la misma ventana), menores que el efecto de cambiar la ventana (ver Figura 6).

Figura 6

Comparación de RMSE, MAPE y R^2 de los nueve modelos en el conjunto común de prueba



Nota. La línea discontinua corresponde al pronóstico de persistencia.

Selección del modelo

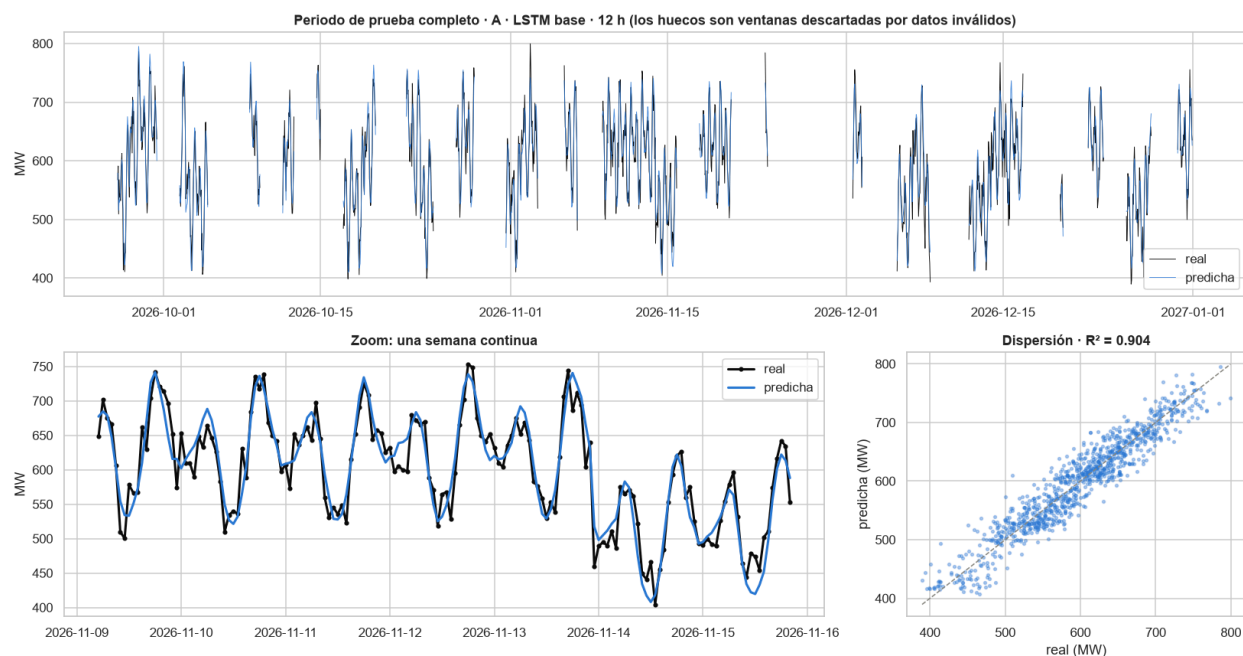
El modelo final se eligió con el RMSE de validación, también calculado sobre instantes comunes, y no con el de prueba. Elegir mirando el conjunto de prueba lo convertiría en otro conjunto de ajuste y la estimación del error dejaría de ser honesta. El modelo seleccionado es la LSTM base (A) con ventana de 12 horas, que tuvo el menor RMSE de validación de los nueve (25,26 MW). En prueba obtiene un MAE de 19,86 MW, un RMSE de 24,99 MW, un MAPE de 3,43 % y un R^2 de 0,904.

Demanda real frente a predicha

Las Figuras 7 y 8 comparan la demanda real con la predicha. El modelo sigue con precisión el ciclo diario, incluidas las subidas de la mañana y de la tarde y los valles nocturnos, y las tres arquitecturas producen curvas casi superpuestas.

Figura 7

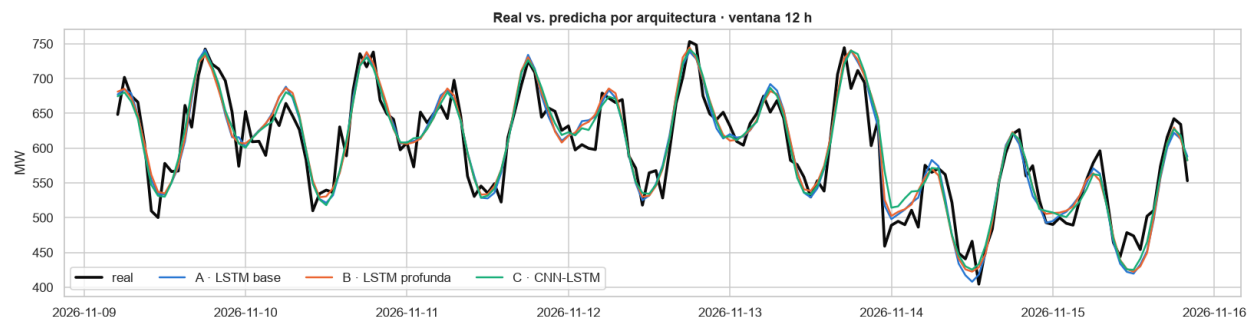
Demanda real y predicha por el modelo seleccionado en el periodo de prueba



Nota. Arriba, el periodo completo (los huecos corresponden a ventanas descartadas por datos inválidos); abajo, una semana continua y el diagrama de dispersión entre valores reales y predichos.

Figura 8

Demanda real y predicciones de las tres arquitecturas durante la misma semana



Nota. Ventana de 12 horas.

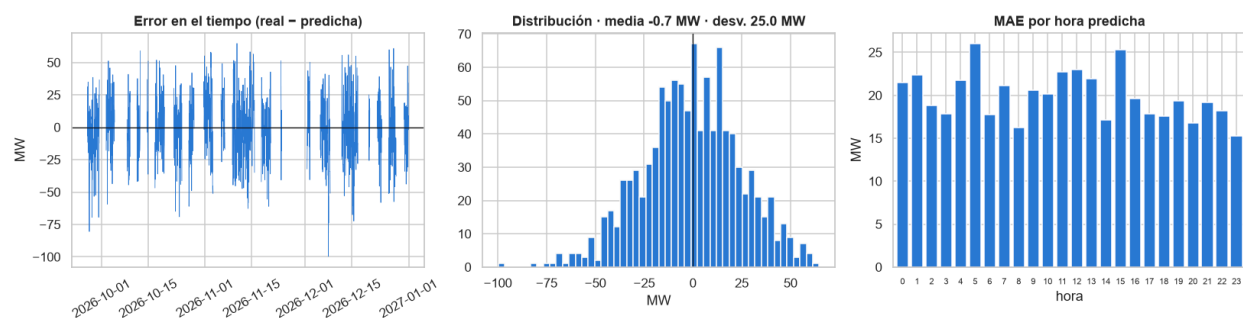
Error de predicción

El error está centrado en cero, con un sesgo medio de $-0,7$ MW, es decir, el modelo no sobrestima ni subestima de forma sistemática. Su distribución es aproximadamente simétrica: el

68,9 % de las horas tiene un error absoluto menor de 25 MW y el percentil 95 es de 49 MW. No hay una franja horaria con errores sistemáticamente mayores (ver Figura 9).

Figura 9

Error de predicción del modelo seleccionado



Nota. De izquierda a derecha: error en el tiempo, distribución del error y error absoluto medio según la hora que se predice.

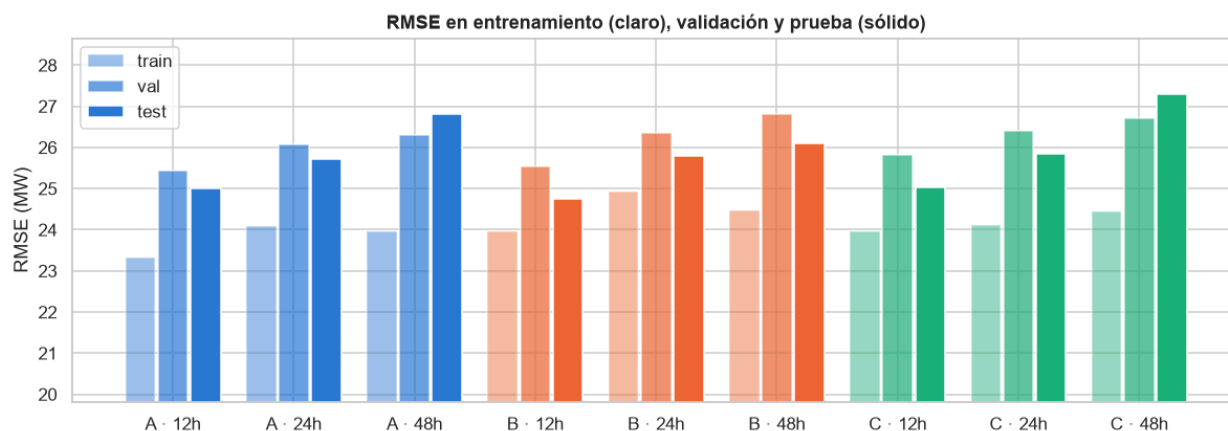
Sobreajuste y generalización

La Tabla 10 y la Figura 10 comparan el error de cada modelo en entrenamiento, validación y prueba.

Tabla 10*RMSE de cada modelo en entrenamiento, validación y prueba*

Modelo	Ventana	Entrenamiento	Validación	Prueba	Prueba / entrenamiento
A. LSTM base	12 h	23,32	25,43	25,01	1,07
A. LSTM base	24 h	24,08	26,08	25,71	1,07
A. LSTM base	48 h	23,96	26,31	26,80	1,12
B. LSTM profunda	12 h	23,95	25,53	24,74	1,03
B. LSTM profunda	24 h	24,92	26,36	25,78	1,03
B. LSTM profunda	48 h	24,46	26,81	26,10	1,07
C. CNN-LSTM	12 h	23,95	25,81	25,03	1,04
C. CNN-LSTM	24 h	24,12	26,41	25,83	1,07
C. CNN-LSTM	48 h	24,45	26,70	27,29	1,12

Nota. RMSE en MW, calculado sobre las ventanas válidas de cada conjunto. La última columna es el cociente entre el RMSE de prueba y el de entrenamiento.

Figura 10*RMSE en entrenamiento, validación y prueba por modelo*

Nota. El tono claro corresponde a entrenamiento y el sólido, a prueba.

El cociente entre el RMSE de prueba y el de entrenamiento está entre 1,03 y 1,12: el error fuera de la muestra es solo de 3 a 12 % mayor que dentro. Por tanto, no hay sobreajuste relevante y los modelos generalizan a un periodo (septiembre a diciembre de 2026) que nunca vieron. Los modelos con Dropout (B y C) tienen las brechas más bajas, y las ventanas de 48 horas las más

altas, porque disponen de menos muestras para la misma cantidad de parámetros. Validación y prueba quedan muy cerca, lo que indica que la validación fue un buen criterio para seleccionar el modelo.

Experimentos de control

Para responder con evidencia las preguntas sobre Dropout, detención temprana y número de neuronas, se entrenaron variantes con la ventana de 12 horas, cambiando un solo factor a la vez. Los resultados aparecen en la Tabla 11 y en la Figura 11.

Tabla 11

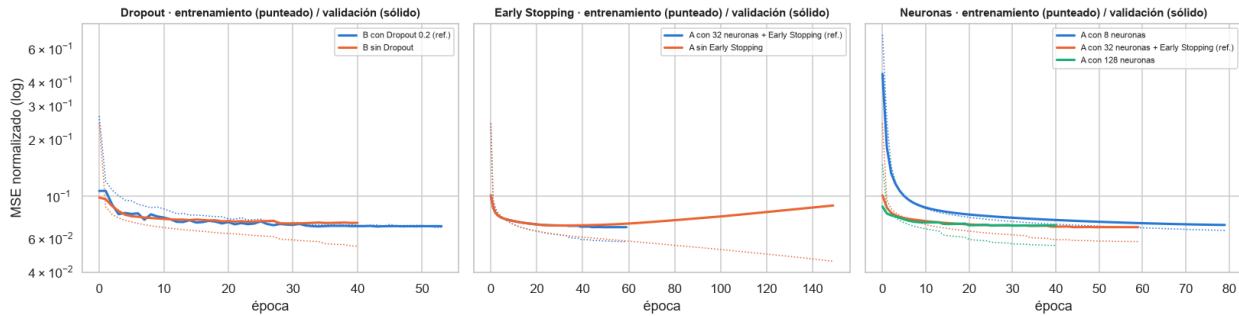
Resultados de los experimentos de control

Variante	Parámetros	Épocas	RMSE entrenamiento	RMSE prueba	R ² prueba	Brecha
A con 32 neuronas + Early Stopping (referencia)	6 177	60	23,32	25,01	0,9076	1,69
B con Dropout 0,2 (referencia)	33 441	54	23,95	24,74	0,9096	0,79
B sin Dropout	33 441	41	23,23	25,26	0,9058	2,03
A sin Early Stopping	6 177	150	21,08	28,87	0,8769	7,79
A con 8 neuronas	777	80	24,80	25,22	0,9061	0,42
A con 128 neuronas	73 857	41	22,88	24,92	0,9083	2,04

Nota. Ventana de 12 horas. RMSE en MW. La brecha es la diferencia entre el RMSE de prueba y el de entrenamiento.

Figura 11

Curvas de entrenamiento y validación de los experimentos de control



Nota. Las curvas punteadas corresponden a entrenamiento y las sólidas, a validación.

- *Sin detención temprana* (150 épocas fijas), la pérdida de validación empieza a subir alrededor de la época 35 mientras la de entrenamiento sigue bajando. El RMSE de prueba empeora de 25,0 a 28,9 MW (un 15 % más), un caso claro de sobreajuste.
- *Sin Dropout* en la arquitectura B, la brecha entre prueba y entrenamiento pasa de 0,8 a 2,0 MW y el RMSE de prueba sube de 24,7 a 25,3 MW.
- *Con distinto número de neuronas*, el RMSE de prueba es de 25,2 MW con 8 neuronas (777 parámetros), de 25,0 MW con 32 y de 24,9 MW con 128 (73 857 parámetros, 95 veces más). Multiplicar los parámetros casi no mejora el resultado y amplía la brecha de generalización.

Evidencia adicional

Comparación con modelos de referencia

Para medir el aporte de la LSTM, se comparó con la persistencia y con una regresión lineal regularizada (Ridge) que recibe exactamente la misma ventana de 12 horas aplanada, es decir, 180 entradas (ver Tabla 12).

Tabla 12*Comparación de la LSTM con modelos de referencia*

Modelo	MAE	MSE	RMSE	MAPE	R ²
Persistencia	35,50	1 963,3	44,31	6,07 %	0,6974
Regresión lineal (Ridge, 12 h)	22,53	874,2	29,57	3,90 %	0,8652
LSTM A (12 h)	19,86	624,6	24,99	3,43 %	0,9037

Nota. Conjunto común de prueba. MAE, MSE y RMSE en MW.

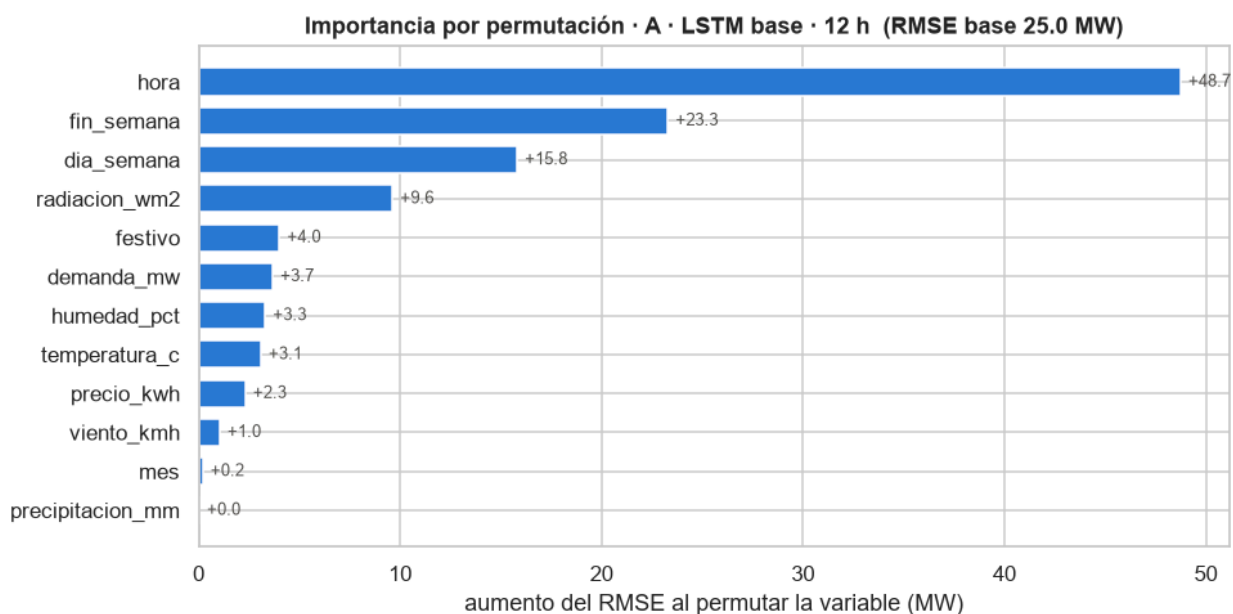
La LSTM reduce el RMSE en un 15 % frente a la regresión lineal y en un 44 % frente a la persistencia. La diferencia con el modelo lineal corresponde a lo que aportan la memoria y la no linealidad de la red.

Importancia de las variables

Se midió la importancia por permutación (Breiman, 2001): se desordenan los valores de una variable entre las muestras de prueba y se observa cuánto aumenta el RMSE del modelo seleccionado (ver Figura 12).

Figura 12

Importancia de las variables por permutación en el modelo seleccionado



Nota. Aumento del RMSE (MW) al permutar cada variable; las codificaciones seno y coseno de una misma variable se permutan juntas.

Dónde se equivoca más el modelo

La Tabla 13 muestra el error del modelo según el tipo de día que se predice.

Tabla 13

Error absoluto medio del modelo seleccionado según el tipo de día

Hora predicha	MAE (MW)	Horas en prueba
Festivo	34,84	38
No festivo	19,64	2 055
Fin de semana	20,01	621
Día laboral	19,87	1 472

Nota. Conjunto de prueba con ventana de 12 horas.

Preguntas de análisis

¿Por qué LSTM es apropiada para este problema?

La demanda eléctrica es una serie temporal: el valor de la próxima hora depende de lo ocurrido en las horas anteriores, por la inercia del consumo, las rampas de la mañana y de la tarde y el ciclo diario. Una red densa trata cada entrada como independiente y no tiene noción de orden. Una red recurrente simple sí la tiene, pero sufre el desvanecimiento del gradiente y olvida lo ocurrido hace muchas horas (Goodfellow et al., 2016).

La LSTM resuelve este problema con una celda de memoria controlada por tres compuertas (Hochreiter y Schmidhuber, 1997):

- la de olvido, que decide qué información descartar;
- la de entrada, que decide qué información nueva incorporar;
- la de salida, que decide qué exponer en cada paso.

Además, acepta entradas multivariadas (demanda, clima, precio y calendario a la vez), aprende relaciones no lineales entre ellas y lee la ventana en orden. Así puede usar la forma de la curva reciente, si viene subiendo o bajando, y no solo su nivel.

La evidencia lo confirma: con la misma ventana, la LSTM obtiene un RMSE de 25,0 MW, frente a 29,6 MW de una regresión lineal y 44,3 MW de la persistencia.

¿Qué efecto tiene cambiar la ventana temporal?

En las tres arquitecturas, la ventana de 12 horas fue la mejor y la de 48 horas la peor (ver Tabla 14).

Tabla 14

RMSE en el conjunto común de prueba según la arquitectura y la ventana

Arquitectura	12 horas	24 horas	48 horas
A. LSTM base	25,0	25,5	26,8
B. LSTM profunda	24,8	25,7	26,1
C. CNN-LSTM	25,3	25,5	27,3

Nota. Valores en MW.

Cambiar la ventana implica un compromiso:

- *Ventana más larga.* Ofrece más contexto, pero implica más entradas, un gradiente que debe propagarse por más pasos y menos muestras de entrenamiento: 9 686 con 12 horas frente a 4 504 con 48 horas, por las ventanas descartadas. El resultado es una peor generalización: el cociente entre prueba y entrenamiento sube de 1,07 a 1,12 en A y C.
- *Ventana más corta.* Ofrece menos contexto, pero en este caso no hace falta más: el ciclo diario ya llega al modelo mediante la codificación cíclica de la hora y del día, y el nivel actual de la demanda está en las últimas horas.

Una ventana demasiado corta perdería la tendencia reciente. La longitud óptima depende del problema y debe elegirse con validación, como se hizo en este trabajo.

¿Existe sobreajuste?

No hay sobreajuste relevante en los modelos finales:

- El RMSE de prueba es solo de 3 a 12 % mayor que el de entrenamiento.
- Las curvas de validación se estabilizan, sin la forma de U característica del sobreajuste.
- La arquitectura A, sin regularización, muestra solo una brecha pequeña y controlada.

El riesgo, sin embargo, existía: la misma arquitectura A entrenada durante 150 épocas sin detención temprana sí sobreajusta. Su RMSE de entrenamiento baja a 21,1 MW, pero el de prueba sube a 28,9 MW. El sobreajuste se evitó gracias a las decisiones de entrenamiento.

¿Qué es y qué efecto tiene Dropout?

Dropout es una técnica de regularización que, durante el entrenamiento, apaga aleatoriamente una fracción p de las neuronas en cada paso (aquí, $p = 0,2$) y escala las restantes para compensar. Al predecir se desactiva y se usa la red completa (Srivastava et al., 2014). Tiene tres efectos:

- Evita que las neuronas dependan de otras específicas (co-adaptación) y obliga a la red a aprender representaciones redundantes y robustas.
- Equivale a entrenar un conjunto de muchas subredes que comparten pesos y a promediarlas al predecir.
- Reduce el sobreajuste, a cambio de un entrenamiento algo más lento y ruidoso.

En el experimento de control, quitar el Dropout de la arquitectura B amplió la brecha entre prueba y entrenamiento de 0,8 a 2,0 MW y empeoró el RMSE de prueba de 24,7 a 25,3 MW. Esto también explica por qué en B y C la pérdida de validación queda por debajo de la de entrenamiento.

¿Qué es y qué efecto tiene Early Stopping?

La detención temprana (*early stopping*) vigila una métrica de validación, aquí la pérdida, y detiene el entrenamiento cuando deja de mejorar durante un número de épocas definido por la paciencia, que en este trabajo es de 10 (Prechelt, 1998). Al restaurar los mejores pesos, el modelo final conserva los de la mejor época y no los de la última. Tiene dos efectos:

- Evita el sobreajuste, porque corta el entrenamiento justo cuando la red empieza a memorizar el ruido.
- Elige automáticamente el número de épocas (entre 42 y 66 en este trabajo) y ahorra cómputo.

Sin detención temprana, la pérdida de validación sube a partir de la época 35 aproximadamente y el RMSE de prueba empeora de 25,0 a 28,9 MW.

¿Más neuronas implican necesariamente mejores resultados?

No. Dos comparaciones lo muestran:

- *Experimento de neuronas* (arquitectura A, ventana de 12 horas). El RMSE de prueba es de 25,2 MW con 8 neuronas, de 25,0 MW con 32 y de 24,9 MW con 128. Multiplicar los parámetros por 95 solo mejora 0,3 MW, y la brecha de generalización crece de 0,4 a 2,0 MW.

- *Comparación entre arquitecturas.* B tiene cinco veces más parámetros que A y queda prácticamente empatada con ella.

Más capacidad solo ayuda mientras el modelo esté subajustado. Una vez que capta los patrones reales, el error restante es ruido impredecible, y la capacidad adicional se usa para memorizarlo, es decir, produce sobreajuste. Además, cuesta más cómputo y más datos (Goodfellow et al., 2016).

¿Qué modelo recomendaría para producción y por qué?

Se recomienda la LSTM base (A) con ventana de 12 horas, por cuatro razones:

1. Tuvo el menor error en validación, que es el criterio correcto de selección. En prueba obtuvo un MAE de 19,9 MW, un RMSE de 25,0 MW, un MAPE de 3,4 % y un R^2 de 0,90.
2. Es el modelo más simple y ligero (6 177 parámetros): su inferencia es instantánea y es fácil de reentrenar, mantener y explicar. Gracias a eso pudo desplegarse sin TensorFlow.
3. B no justifica su complejidad: fue 0,2 MW mejor en prueba, una diferencia dentro del ruido que no compensa tener cinco veces más parámetros. Ante un empate, la parsimonia favorece al modelo simple.
4. La ventana de 12 horas requiere poca historia, así que en operación es menos probable que un dato faltante impida la predicción que con 24 o 48 horas.

En producción, el modelo debería acompañarse de monitoreo del error, de la validación de las entradas con las mismas reglas de limpieza y de un reentrenamiento periódico.

¿Qué variables pueden influir más en la demanda?

Según la importancia por permutación, el modelo depende sobre todo del calendario: la hora del día (+48,7 MW de RMSE al permutarla), el fin de semana (+23,3 MW) y el día de la semana (+15,8 MW). Les siguen:

- la radiación solar (+9,6 MW);
- los festivos (+4,0 MW);

- la demanda reciente (+3,7 MW);
- la humedad (+3,3 MW) y la temperatura (+3,1 MW);
- el precio (+2,3 MW).

El viento, el mes y la precipitación casi no aportan. Esto es coherente con el análisis exploratorio, que mostró un perfil diario marcado, una demanda menor en fin de semana ($r = -0,49$) y su relación con la temperatura ($r = 0,48$) y la radiación ($r = -0,50$).

Hay dos matices:

- La importancia por permutación no implica causalidad, y las variables correlacionadas se reparten la importancia; por ejemplo, la radiación depende de la hora.
- La demanda reciente aparece más abajo de lo que su correlación sugiere (0,88), porque gran parte de su información ya está contenida en el calendario y el clima.

¿Qué limitaciones tiene el modelo?

- *Festivos*. El error medio en horas festivas es de 34,8 MW, casi el doble que en los días normales (19,6 MW). El entrenamiento solo incluye 8 días festivos (192 horas).
- *Datos faltantes en operación*. Como no se imputa, si alguna de las últimas 12 horas tiene un valor inválido no se puede predecir. Un sistema real necesitaría un plan de contingencia.
- *Horizonte de una hora*. Para planear el día siguiente habría que predecir de forma recursiva, con acumulación de errores, o entrenar un modelo de varios horizontes.
- *Sin pronósticos exógenos*. El modelo usa el clima observado hasta t , no pronósticos meteorológicos de $t + 1$.
- *Historia corta*. Dos años de datos implican pocos ciclos de cada estación, y la prueba solo cubre de septiembre a diciembre.
- *Cambios bruscos*. Cerca del 5 % de las horas tiene errores mayores de 50 MW, en general en cambios abruptos que el modelo suaviza.

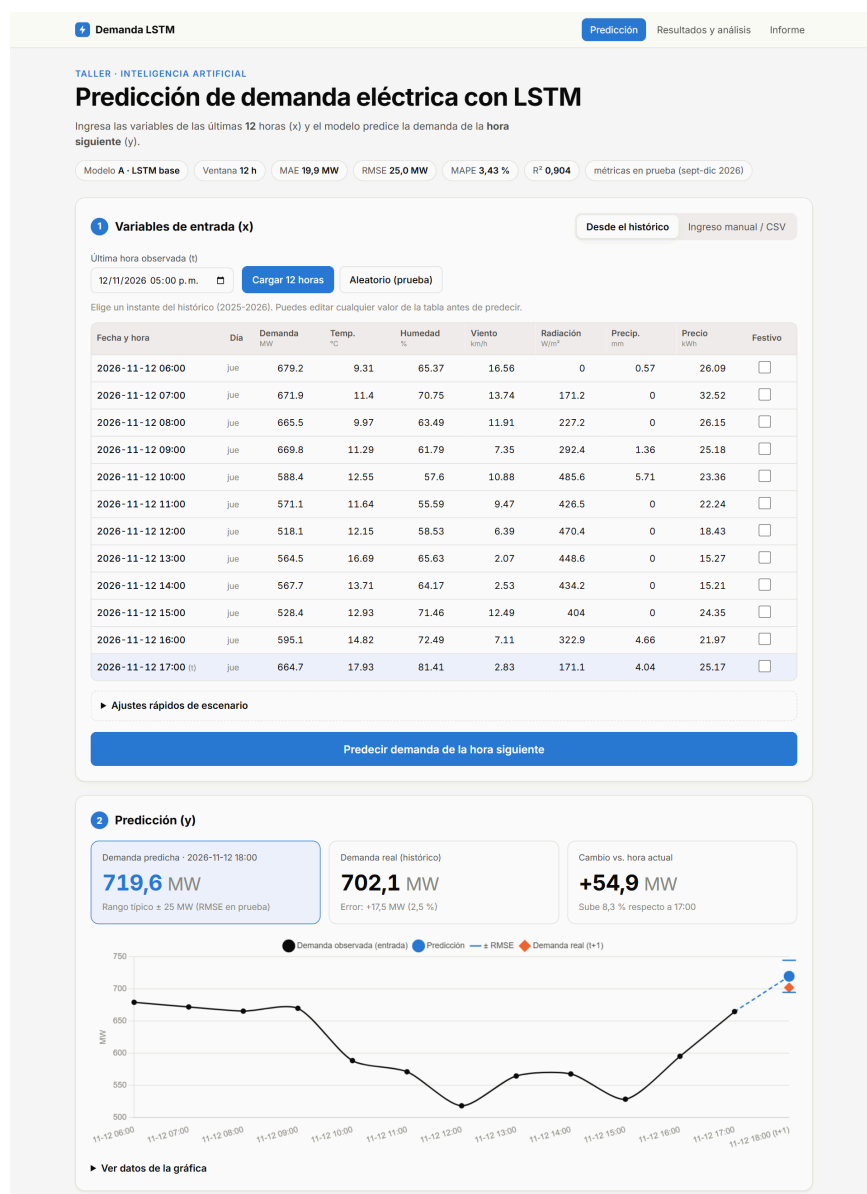
- *Predicción puntual.* El modelo no entrega un intervalo de confianza.
- *Deriva de los datos.* Si cambian los patrones de consumo, el modelo debe reentrenarse.

Despliegue

El modelo seleccionado se desplegó como una aplicación web pública en Vercel (<https://red-neuronal-app.vercel.app>). Su página de predicción aparece en la Figura 13.

Figura 13

Página de predicción de la aplicación web desplegada



Nota. Captura de la aplicación en funcionamiento con una ventana de 12 horas del periodo de prueba.

En la aplicación, el usuario ingresa las variables independientes (x) de las últimas 12 horas: puede cargarlas desde el histórico, escribirlas a mano o subir un archivo CSV. La aplicación muestra:

- la predicción de la demanda de la hora siguiente (y), con su rango típico de error ($\pm$ RMSE);
- una gráfica con la demanda reciente;
- el valor real para comparar, cuando existe.

También permite simular escenarios, como subir la temperatura, cambiar la radiación o marcar un festivo. Otras dos páginas presentan los resultados del taller y este informe.

La aplicación tiene tres componentes técnicos:

- *Interfaz*. Está hecha con HTML, CSS y JavaScript estáticos.
- *Servicio de predicción*. Es una función *serverless* en Python (`POST /api/predict`). Valida las entradas con las mismas reglas de limpieza del entrenamiento: si una hora tiene un dato faltante o fuera de rango, informa la fila y el campo exactos y no predice. Si las entradas son válidas, construye las 15 variables por hora, las normaliza con los parámetros de entrenamiento y ejecuta la red.
- *Inferencia sin TensorFlow*. TensorFlow ocupa más de 500 MB y supera el límite de 250 MB de las funciones *serverless*. Como el modelo A tiene solo 6 177 parámetros, sus pesos se exportaron a JSON y el paso hacia adelante de la LSTM se implementó con NumPy. Se verificó sobre 300 ventanas de prueba que esta implementación reproduce las predicciones de Keras con una diferencia máxima de 0,005 MW, atribuible al redondeo a dos decimales.

Conclusiones

Tras una limpieza estricta y justificada que no inventa valores, y con una preparación cronológica sin fuga de información, las LSTM predicen la demanda de la hora siguiente con un MAPE cercano a 3,4 % y un R^2 cercano a 0,90 en un periodo nunca visto. Esto supone un 44 % menos de error que la persistencia y un 15 % menos que una regresión lineal.

La ventana de 12 horas fue la mejor en las tres arquitecturas: las ventanas más largas aportan poco contexto adicional y reducen las muestras disponibles. Las diferencias entre

arquitecturas fueron pequeñas, de modo que la complejidad adicional de la LSTM profunda y de la CNN-LSTM no produjo mejoras relevantes. Por su simplicidad, ligereza y robustez, se recomienda la LSTM base.

Dropout y la detención temprana fueron determinantes para evitar el sobreajuste, como demostraron los experimentos de control, y aumentar el número de neuronas no mejoró los resultados. La demanda depende principalmente del calendario (hora, fin de semana y día de la semana) y, en segundo lugar, del clima y de la demanda reciente. El principal punto débil del modelo son los días festivos.

Finalmente, el modelo se desplegó en una aplicación web que permite ingresar las variables y obtener la predicción, respetando las mismas reglas de calidad de datos del entrenamiento.

Referencias

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.
<https://doi.org/10.1023/A:1010933404324>
- Chollet, F., et al. (2015). *Keras* [Software]. <https://keras.io>
- Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep learning*. MIT Press.
- Hochreiter, S., y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Hyndman, R. J., y Athanasopoulos, G. (2021). *Forecasting: Principles and practice* (3.^a ed.). OTexts. <https://otexts.com/fpp3/>
- Kingma, D. P., y Ba, J. (2015). Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1412.6980>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., y Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Prechelt, L. (1998). Early stopping - but when? En G. B. Orr y K.-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 55-69). Springer. https://doi.org/10.1007/3-540-49430-8_3
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., y Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929-1958. <https://jmlr.org/papers/v15/srivastava14a.html>